HAC: Simple Hybrids, Smooth Schedules — Rethinking Adaptive Fusion for Resilient UAV Swarm Connectivity Recovery

Abstract

Self-healing connectivity recovery for unmanned aerial vehicle (UAV) swarms requires restoring network connectivity rapidly after partial node failures while preserving wide communication coverage. Peng et al. (2026) recently addressed this trade-off with R2C, combining an intra-subnetwork virtual-force expansion module, a subnetwork-level multipartite graph convolution module, and a time-adaptive weighted fusion schedule (AWF), yet bundling these components obscures the individual roles of spatial reasoning granularity and schedule transition steepness. We systematically ablate this design space across decision paradigm, spatial reasoning granularity, and connectivity–coverage trade-off strategy through controlled multi-seed experiments. We propose Hybrid Adaptive Convolution (HAC), which fuses a parameter-free virtual-force heuristic branch with a single global graph convolution branch at the velocity level while omitting subnetwork partitioning, convex-hull boundary extraction, and multipartite graph construction. Across random and uniform deployments, fixed-weight HAC boosts the 80%-failure recovery rate by up to 36.8 percentage points (pp) over single-branch learned baselines and retains 88.7% to 90.7% of R2C's reported spatial coverage (within 6.2 to 7.7 pp). Furthermore, we show that transplanting R2C's steep adaptive schedule directly into HAC degrades high-damage recovery, whereas smoothing the transition slope restores robustness and slightly improves coverage. We prove that the heuristic branch guarantees local distance descent in the attractive zone and explain how the fusion schedule acts as an implicit training curriculum governing high-autonomy exposure. Finally, a subnetwork-level hybrid counter-experiment demonstrates that schedule smoothing cannot rescue policies whose rigid translation output space lacks node-level expressiveness.

1 Introduction

1.1 Problem and Motivation

Partial node failures fracture swarm topologies and threaten mission continuity. Unmanned aerial vehicle (UAV) swarms provide rapid, flexible wireless coverage in post-disaster relief, dynamic battlefields, and congested areas (Coletta et al., 2023; Shi et al., 2025; Wang et al., 2022, 2024b). During operation, internal faults or external attacks can disable a subset of UAVs at any time (Wang et al., 2024a; Kurunathan et al., 2023). When substantial node losses occur, the communication graph fractures into disconnected subnetworks, sharply degrading both communication reliability and spatial coverage across the mission area (Li et al., 2023; Xu et al., 2022; Peng et al., 2022). The self-healing communication connectivity (SCC) problem seeks a distributed or partially observed velocity control policy that repositions surviving UAVs to restore full graph connectivity as rapidly as possible while maximizing spatial coverage.

Connectivity restoration and spatial coverage preservation are in direct physical tension. Pulling surviving UAVs toward the swarm center accelerates reconnection among fragmented components but shrinks the covered footprint, whereas spreading nodes outward preserves area coverage at the risk of slowing or preventing reconnection. Balancing these competing forces becomes especially challenging when large failure ratios leave small, widely separated subnetworks across the mission space.

1.2 Limitations of Existing Approaches

Hand-crafted heuristic rules fail to scale under severe, unstructured failures. Non-learning SCC algorithms either identify and replace critical cut-vertices through cascaded neighbor movements (Akkaya et al., 2009; Mi and Yang, 2015; Zhang et al., 2024) or guide surviving nodes toward predefined rendezvous points using potential fields (Joshi, 2012; Shriwastav, 2018; Chen et al., 2020; Wang et al., 2024c). Although heuristic rules require no training data and preserve local spacing well, their static coordination assumptions break down when severe node losses disconnect components beyond local sensing ranges (Tosun et al., 2024).

Purely learned graph policies suffer from spatial collapse or rigid-body expressiveness limits. Graph neural network (GNN) methods formulate SCC as a learned graph optimization task and reconnect fragmented topologies faster than heuristics (Mou et al., 2022; Peng et al., 2025; Wang et al., 2025b). However, message-passing aggregation biases surviving UAVs to cluster around high-degree nodes, causing severe spatial collapse and coverage loss. Diffusion-based and differential-subgraph convolutions (Lin et al., 2024; Lin and Ding, 2025) encourage broader dispersion, yet struggle to maintain stable layouts without explicit local repulsion.

1.3 Our Approach and Findings

Isolating spatial granularity and schedule steepness reveals what makes hybrid recovery work. Peng et al. (2026) recently introduced R2C, which combines a buffered virtual-force field for intra-subnetwork expansion (ISE) with a multipartite graph convolution module for inter-subnetwork translation (IST) under an adaptive weighted fusion (AWF) schedule. While R2C achieves strong empirical performance, its module-level ablations toggle entire branches on and off without isolating whether the learned component must operate at subnetwork granularity or whether AWF requires its specific steep sigmoid transition. To answer both questions, we introduce **HAC** (Hybrid Adaptive Convolution), a streamlined architecture that fuses R2C's parameter-free virtual-force branch at the velocity level with a single global k-nearest-neighbor (k-NN) graph convolution branch, eliminating subnetwork partitioning, convex-hull boundary extraction, and multipartite virtual edges.

Controlled experiments across random and uniform deployments show that fixed-weight global hybridization (**HAC-Fixed**) achieves near-perfect connectivity recovery across all tested failure rates and retains the vast majority of R2C's reported coverage under extreme node loss with substantially lower variance than single-branch policies. Conversely, directly applying R2C's steep adaptive schedule to HAC degrades recovery under severe failures. Our schedule sweep demonstrates that the bottleneck is the transition steepness rather than the handover timing, because an overly steep schedule deprives the learned branch of residual-correction supervision during differentiable rollout training. Smoothing the sigmoid slope (**HAC-Smooth**) restores high-failure recovery rate while improving coverage and node-degree uniformity. Substituting a subnetwork-level rigid-translation branch (**HSC**) confirms that schedule smoothing only succeeds when the learned policy possesses node-level velocity expressiveness.

1.4 Contributions

1. **Three-axis design taxonomy (§3, §4).** We categorize SCC methods along decision paradigm, spatial reasoning granularity, and connectivity–coverage trade-off strategy, and demonstrate through motivating baselines that purely global learned policies (GAC) suffer from spatial clustering whereas purely subnetwork-level rigid policies (SAC) fail deterministically under high node loss.
2. **Minimalist global hybrid architecture (§5, §8).** We propose HAC, fusing a parameter-free virtual-force branch with a global k-NN graph convolution branch. Five-seed evaluations across two deployment geometries show that HAC-Fixed and HAC-Smooth outperform single-branch learned and heuristic baselines by wide margins under severe failures while retaining 88.7% to 90.7% of R2C's published coverage without boundary-hull or multipartite machinery.
3. **Theoretical and empirical diagnosis of adaptive fusion schedules (§6, §9).** We prove a local attractive-zone descent guarantee for the heuristic branch and formulate an autonomy-exposure curriculum framework showing why steep fusion transitions impair end-to-end training, validated via schedule steepness sweeps and a subnetwork-level hybrid counter-experiment (HSC).
4. **Count-invariant connectivity loss and reproducible benchmark (§5, Appendix A).** We identify and correct a pair-count bias in log-sum-exp soft-minimum connectivity surrogates, document the impact of weight-initialization seeding on multi-seed variance, and release the complete simulation and training suite.

2 Related Work

2.1 Non-Learning Self-Healing Methods

Critical-node compensation scales poorly with failure density. Early topological recovery schemes detect cut-vertices whose removal partitions the network and trigger cascaded neighbor movements to fill the resulting coverage gaps (Abbasi and Younis, 2008; Akkaya et al., 2009; Mi and Yang, 2015; Zhang et al., 2024). While effective for isolated single-node faults, identifying critical nodes and coordinating multi-hop cascades incur high signaling overhead that fails when simultaneous failures eliminate multiple adjacent relays.

Rendezvous-point aggregation struggles under uncertain fragmentation. Alternative heuristic methods guide surviving UAVs or disconnected clusters toward predetermined rendezvous coordinates using Coulomb-force models (Joshi, 2012), round-table negotiation (Shriwastav, 2018), or artificial potential fields (Chen et al., 2020; Wang et al., 2024c). Although parameter-free and computationally lightweight, fixed attraction rules cannot adapt when surviving subnetworks are separated beyond their local communication horizon (Tosun et al., 2024).

2.2 Learning-Based Self-Healing Methods

Relaxed GNN and attention policies accelerate reconnection but induce spatial collapse. Casting SCC as a differentiable graph optimization problem enables data-driven policies to reason over long-range topological structure (Karalias and Loukas, 2020; Peng et al., 2025). Mou et al. (2022) apply a relaxed graph convolutional network over virtual links, and Guo et al. (2024) incorporate spatiotemporal multi-head attention. Despite faster reconnection than heuristics, message aggregation pulls surviving UAVs tightly toward high-degree hubs, causing severe spatial clustering and coverage loss.

Diffusion and differential-subgraph convolutions lack explicit local collision buffering. Recent learning architectures introduce multi-hop diffusion (Lin et al., 2024) or bipartite convolution over differential subgraphs (Lin and Ding, 2025) to promote dispersed recovery. While these designs improve post-recovery spread compared to standard GCNs, they do not enforce hard inter-agent spacing zones during transit.

2.3 Hybrid Heuristic–Learning Architectures and Positioning

Prior hybrid frameworks either fix node-level weights or bundle subnetwork modules with steep schedules. Li et al. (2025) combine distributed multi-agent reinforcement learning with an artificial potential field at node-level granularity under fixed weighting, whereas Wang et al. (2025b) rely solely on graph reinforcement learning. Related multi-robot coverage and spectral GNN studies explore substructure-aware message passing (Kipf and Welling, 2016; Zeng et al., 2023; Liu and Wang, 2025; Mutzari et al., 2025; Ye et al., 2022). Peng et al. (2026) advance this line with R2C, pairing buffered virtual forces for intra-subnetwork expansion (ISE) with convex-hull boundary extraction (Barber et al., 1996) and Chebyshev multipartite graph convolution for inter-subnetwork translation (IST), blended via a sigmoid adaptive weighted fusion (AWF) schedule (Elfwing et al., 2018). Because R2C's ablations only remove entire modules, the literature lacks a controlled isolation of spatial reasoning granularity (global vs. subnetwork learned branches) and schedule transition steepness. Our work directly bridges this gap.

3 Preliminaries and Problem Formulation

3.1 UAV Swarm Network Model

Swarm connectivity is governed by the algebraic multiplicity of the communication Laplacian. Following Peng et al. (2026), at discrete time step t the surviving UAV swarm forms a temporal proximity graph $G_t = (V_t, E_t, P_t)$, where $V_t = \{v_{1,t}, \ldots, v_{|V_t|,t}\}$ denotes the surviving UAVs, $P_t = \{\mathbf{p}_{1,t}, \ldots, \mathbf{p}_{|V_t|,t}\}$ with $\mathbf{p}_{i,t} \in \mathcal{A} \subset \mathbb{R}^2$ denotes their planar coordinates within mission area $\mathcal{A}$, and E_t denotes active communication links. A direct air-to-air link (Ma et al., 2019; Wang et al., 2025a) exists between v_i and v_j if and only if their Euclidean distance $d_{ij,t} = \|\mathbf{p}_{i,t} - \mathbf{p}_{j,t}\|_2$ satisfies $d_{ij,t} \leq R_c$, where the communication radius R_c follows a free-space path-loss model (Ni et al., 2024). Let $\mathbf{A}_t$ be the adjacency matrix and $\mathbf{L}_t = \mathbf{D}_t - \mathbf{A}_t$ be the graph Laplacian. The number of connected components C_t equals the multiplicity of the zero eigenvalue of $\mathbf{L}_t$, so the swarm is fully connected if and only if $C_t = 1$. Each UAV also covers a disk $\mathcal{C}_i(t)$ of radius R_{cov}, yielding the collective area coverage ratio $C_{\text{cov}}(t) = |\bigcup_{i \in V_t} \mathcal{C}_i(t)|/|\mathcal{A}|$.

3.2 Problem Definition

The SCC objective jointly penalizes normalized reconnection time and post-recovery coverage loss. Let t_1 denote the instant of node failure and $t_r \leq t_1 + T$ denote the earliest subsequent step at which $C_{t_r} = 1$ within horizon T. The normalized recovery time ratio is $T_{\text{rec}} = (t_r - t_1)/T$. The control policy generates bounded per-node velocities $\hat{V}_t = \{\hat{\mathbf{v}}_{1,t}, \ldots, \hat{\mathbf{v}}_{|V_t|,t}\}$ to minimize

$$J_c = T_{\text{rec}} + \big(1 - C_{\text{cov}}(t_r)\big), \tag{1}$$

subject to four operational constraints: (C1) terminal connectivity $C_{t_r} = 1$ within horizon T, where trials failing to reconnect by $t_1 + T$ are marked unsuccessful and evaluated primarily via recovery success rate; (C2) speed and boundary limits $\|\hat{\mathbf{v}}_{i,t}\|_2 \leq \hat{v}_{\max}$ and $\mathbf{p}_{i,t} \in \mathcal{A}$; (C3) discrete kinematics

Table 1: Taxonomy of the $3 \times 3 \times 3$ SCC design space positioning prior methods and our four controlled architectures (GAC, SAC, HAC, and HSC).

Paradigm	Granularity	Trade-off	Representative method(s)
Heuristic	Node-level	Fixed	Akkaya et al. (2009); Mi and Yang (2015); Wang et al. (2024c)
Heuristic	Subnetwork	Fixed	Chen et al. (2020)
Heuristic	Global	Fixed	Lin and Ding (2025)
Heuristic	Global	Staged	HERO (Mi et al., 2011)
Learned	Node-level	Fixed	Mou et al. (2022)
Learned	Global	Fixed	Guo et al. (2024); **GAC (this work, §4)**
Learned	Subnetwork	Fixed	Lin et al. (2024); **SAC (this work, §4)**
Hybrid	Node-level	Fixed	Li et al. (2025)
Hybrid	Node+Subnet	Adaptive	R2C (ISE+IST+AWF) (Peng et al., 2026)
Hybrid	**Global**	**Fixed / Adaptive**	**HAC-Fixed / HAC-Smooth (this work, §5)**
Hybrid	**Subnetwork**	**Fixed / Adaptive**	**HSC (this work, §9)**

$\mathbf{p}_{i,t+1} = \mathbf{p}_{i,t} + \Delta t \cdot \hat{\mathbf{v}}_{i,t}$; and (C4) link feasibility $d_{ij,t} \leq R_c$. Minimizing T_{rec} pulls components inward whereas minimizing $1 - C_{\text{cov}}(t_r)$ pushes nodes outward.

3.3 A Taxonomy of the Design Space

Three orthogonal design axes organize prior SCC methods and our controlled ablations. Every self-healing controller must specify three architectural choices:

- **Axis 1 (Decision Paradigm):** Whether motion commands originate from parameter-free physical or graph rules (*heuristic*), a trained parametric network (*learned*), or a velocity-level blend of both (*hybrid*).
- **Axis 2 (Spatial Reasoning Granularity):** Whether the controller reasons over 1-hop communication neighbors (*node-level*), treats each connected component as a rigid unit (*subnetwork-level*), or passes messages across the entire surviving swarm (*global-level*).
- **Axis 3 (Trade-off Strategy):** Whether connectivity and coverage priorities are combined with constant weights (*fixed*), executed in strict temporal phases (*staged*), or continuously re-weighted over time (*adaptive*).

Table 1 maps representative literature and the four experimental architectures studied in this work into this $3 \times 3 \times 3$ taxonomy: **GAC** (global learned policy), **SAC** (subnetwork-level rigid-translation learned policy), **HAC** (our proposed hybrid combining node-level heuristic forces with a global learned GCN under fixed or smooth adaptive schedules), and **HSC** (a hybrid counter-experiment pairing the same heuristic branch with SAC's subnetwork-level learned branch).

4 Motivating Experiments: Failure Modes of Single-Branch Designs

Single-branch learned policies face a fundamental dilemma between spatial collapse and fragmentation failure. Before introducing our hybrid architecture, we evaluate two pure

Table 2: Motivating single-branch learned baselines under random deployment (mean ± std over 5 seeds, 50 trials per seed). GAC-Fixed exhibits spatial clustering (elevated AND), whereas SAC-Fixed avoids clustering but fails deterministically at 80% node loss.

Failure	GAC-Fixed (global learned)		SAC-Fixed (subnetwork learned)	
rate	AND	Success/50	AND	Success/50
20%	13.68 ± 0.28	49.0 ± 0.6	6.82 ± 0.00	40.0 ± 0.0
40%	12.48 ± 0.61	47.2 ± 1.7	5.85 ± 0.00	26.0 ± 0.0
60%	10.55 ± 0.78	42.8 ± 3.7	4.18 ± 0.00	5.0 ± 0.0
80%	6.00 ± 0.26	19.8 ± 5.5	n/a	0.0 ± 0.0

learned baselines that isolate the spatial granularity axis under identical loss functions and training rollouts:

- **Global Adaptive-weighted Convolution (GAC-Fixed):** A 4-layer GCN operating over a Euclidean k-NN graph ($k = 8$) across all surviving UAVs regardless of communication range R_c, predicting per-node 2D velocities directly.
- **Subnetwork-level Adaptive Convolution (SAC-Fixed):** A 4-layer GCN operating over a k-NN graph of connected-component centroids (pooled from component coordinates and relative sizes), predicting a single rigid 2D translation velocity shared by all UAVs within each subnetwork.

Global message passing causes severe spatial clustering, whereas rigid subnetwork translation collapses under high node loss. Table 2 summarizes five-seed evaluations under random deployment. GAC-Fixed maintains moderate connectivity recovery up to 60% node failure but produces an elevated trajectory-averaged node degree (AND of 13.68 at 20% failure, nearly double SAC-Fixed's 6.82), confirming that unconstrained global GNN aggregation pulls UAVs into dense clusters at the expense of area coverage. Conversely, forcing all nodes within a component to translate rigidly in SAC-Fixed prevents internal clustering and keeps AND low, but destroys recovery capability as fragmentation increases, dropping to zero successful recoveries out of 50 trials at 80% failure across all five seeds. Because isolated single-UAV and small-cluster subnetworks cannot reshape their internal boundaries to bridge wide gaps under rigid translation, neither granularity succeeds alone. This complementary failure pattern directly motivates fusing a local spacing heuristic with a global learned velocity field.

5 Method: Hybrid Adaptive Convolution (HAC)

5.1 Overview and Architectural Minimalism

HAC pairs parameter-free local virtual forces with a single global graph convolution at the velocity level. To test whether R2C's complex subnetwork machinery is necessary, HAC instantiates the *hybrid* × *global* cell of Table 1 in its simplest form. Each surviving UAV's velocity command is synthesized from two parallel branches: a deterministic virtual-force heuristic branch that regulates 1-hop neighbor spacing and boundary avoidance, and a global k-NN GCN branch that provides long-range directional guidance across disconnected components without convex-hull boundary extraction or multipartite virtual edges.

5.2 Heuristic Branch: Buffered Virtual-Force Field

A three-zone virtual spring regulates local spacing and prevents boundary escape without learnable parameters. Adopting R2C's buffered Hookean force model (Rychlewski, 1984; Peng et al., 2026) with neutral zone $[d_l, d_u] = [0.8R_c, 0.9R_c] \subset (0, R_c]$, the pairwise force exerted on UAV v_i by a communication neighbor $v_j \in \mathcal{N}_i = \{j : 0 < d_{ij,t} \le R_c\}$ is

$$\mathbf{F}'_{ij,t} = \begin{cases} -\exp\left(-\alpha \cdot \dfrac{d_{ij,t}}{d_l}\right) \mathbf{u}_{ij,t}, & 0 < d_{ij,t} < d_l, \\ \mathbf{0}, & d_l \le d_{ij,t} \le d_u, \\ \exp\left(-\beta \cdot \dfrac{R_c - d_{ij,t}}{R_c - d_u}\right) \mathbf{u}_{ij,t}, & d_u < d_{ij,t} \le R_c, \end{cases} \tag{2}$$

where $\mathbf{u}_{ij,t} = (\mathbf{p}_{j,t} - \mathbf{p}_{i,t})/d_{ij,t}$ is the unit vector pointing from v_i to v_j, and $\alpha = \beta = 0.01$. To prevent UAVs from drifting outside the mission square $\mathcal{A} = [0, L]^2$, any UAV within buffer distance $d_e = 0.15R_c$ of a boundary edge $b \in \mathcal{B} = \{\text{left, right, bottom, top}\}$ receives an inward repulsion force $\mathbf{F}''_{i,t} = \sum_{b \in \mathcal{B}, 0 \le d_{i,b} < d_e} \exp(-\delta \cdot d_{i,b})\, \mathbf{n}_b$, where $d_{i,b} \ge 0$ is the perpendicular distance to edge b, $\mathbf{n}_b$ is its inward unit normal, and $\delta = 0.01$. Summing over communication neighbors yields $\mathbf{F}_{i,t} = \sum_{j \in \mathcal{N}_i} \mathbf{F}'_{ij,t} + \mathbf{F}''_{i,t}$ and normalized heuristic velocity $\mathbf{v}'_{i,t} = \hat{v}_{\max} \cdot \mathbf{F}_{i,t}/(\|\mathbf{F}_{i,t}\|_2 + \epsilon)$.

5.3 Learned Branch: Global Graph Convolution

A global Euclidean k-NN graph enables cross-component message passing without subnetwork partitioning. At each step t, we construct a symmetric k-NN graph ($k = 8$) over the Euclidean coordinates of all surviving UAVs so that topological information propagates across disconnected components even when inter-component distances exceed R_c. Each UAV v_i is represented by a 3-dimensional feature vector $\mathbf{x}_{i,t} = [p_{i,x}/L,\ p_{i,y}/L,\ \deg_k(v_i)/\max_j \deg_k(v_j)]^\top$, omitting explicit subnetwork IDs so that node representations remain invariant to arbitrary component labelings. Passing $\{\mathbf{x}_{i,t}\}$ through an $L_{\text{gcn}} = 4$ layer GCN (Kipf and Welling, 2016) with hidden dimension 128 and ReLU activations, followed by a two-layer MLP and elementwise tanh scaling by $\hat{v}_{\max}$, produces the learned velocity vector $\mathbf{v}''_{i,t}$.

5.4 Velocity-Level Fusion: Fixed and Smooth Adaptive Schedules

Velocity-level normalization blends local heuristic expansion with global learned translation. Given time horizon H ($H = T_{\text{train}}$ during training and $H = T$ during evaluation), the adaptive fusion schedule assigns weights $w_T(t)$ and $w_E(t)$ via a parameterized sigmoid:

$$w_T(t; k, s_0) = \frac{1}{1 + \exp\big(-k \cdot (t/H - s_0)\big)}, \qquad w_E(t; k, s_0) = 1 - w_T(t; k, s_0), \tag{3}$$

where $k > 0$ governs transition steepness and $s_0 \in (0, 1)$ sets the midpoint fraction. The two branch velocities are fused and renormalized to maximum speed $\hat{v}_{\max}$:

$$\hat{\mathbf{v}}_{i,t} = \hat{v}_{\max} \cdot \frac{w_E(t)\, \mathbf{v}'_{i,t} + w_T(t)\, \mathbf{v}''_{i,t}}{\left\| w_E(t)\, \mathbf{v}'_{i,t} + w_T(t)\, \mathbf{v}''_{i,t} \right\|_2 + \epsilon}. \tag{4}$$

We instantiate two primary variants of our method: **HAC-Fixed**, which sets constant equal weights $w_E(t) = w_T(t) = 0.5$ across all steps, and **HAC-Smooth**, which employs a gentle handover ($k = 5, s_0 = 0.7$) in contrast to R2C's steep default ($k = 30, s_0 = 0.125$).

Algorithm 1 HAC End-to-End Differentiable Rollout Training (one epoch)

Require: Learned policy π_θ, rollout horizon T_{train}, fusion schedule $w_T(\cdot)$

Sample failure ratio $r \in \{0.2, 0.4, 0.6, 0.8\}$; initialize surviving positions $\mathbf{P}_0$
$\mathcal{L}_{\text{total}} \leftarrow 0$
for $t = 0, \ldots, T_{\text{train}} - 1$ **do**
  Partition $\mathbf{P}_t$ into connected components $\{S_m\}$ under communication range R_c
  Compute parameter-free heuristic velocities $\mathbf{v}'_t$ via Eq. (2) ▷ stop gradient
  Construct global Euclidean k-NN graph on $\mathbf{P}_t$ and predict $\mathbf{v}''_t \leftarrow \hat{v}_{\max}\, \pi_\theta(\mathbf{P}_t)$
  Compute fused velocities $\hat{\mathbf{v}}_t$ from $(\mathbf{v}'_t, \mathbf{v}''_t, 1 - w_T(t), w_T(t))$ via Eq. (4)
  $\mathbf{P}_{t+1} \leftarrow \text{clip}(\mathbf{P}_t + \Delta t \cdot \hat{\mathbf{v}}_t,\ 0,\ L)$
  $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + 0.5\, \mathcal{L}_{\text{conn}}(\mathbf{P}_{t+1}) + 0.5\, \mathcal{L}_{\text{cov}}(\mathbf{P}_{t+1}) + \lambda\, \mathcal{L}_{\text{move}}(\mathbf{P}_t, \mathbf{P}_{t+1})$
end for
Backpropagate $\mathcal{L}_{\text{total}}/T_{\text{train}}$ and update θ via Adam with gradient clipping at 1.0

Algorithm 2 HAC Deployment Rollout

Require: Trained policy π_θ, evaluation horizon T, fusion schedule $w_T(\cdot)$

for $t = 0, \ldots, T - 1$ **do**
  if $C_t = 1$ **then return** success at recovery step $t_r = t$
  end if
  Compute $\mathbf{v}'_t$ via Eq. (2) and $\mathbf{v}''_t \leftarrow \hat{v}_{\max}\, \pi_\theta(\mathbf{P}_t)$
  Update $\mathbf{P}_{t+1} \leftarrow \text{clip}\big(\mathbf{P}_t + \Delta t \cdot \text{fuse}(\mathbf{v}'_t, \mathbf{v}''_t, 1 - w_T(t), w_T(t)),\ 0,\ L\big)$
end for
return failure (horizon T exhausted with $C_T > 1$)

5.5 Count-Invariant Differentiable Training Objective

Subtracting log pair counts makes soft-minimum inter-component distance estimation invariant to cluster size. Because the discrete component count C_t is non-differentiable, we train the learned branch end-to-end through T_{train}-step differentiable rollouts using a corrected soft-minimum connectivity surrogate. For every pair of disconnected components (S_i, S_j) at step t with node counts $n_i = |S_i|$ and $n_j = |S_j|$, we define

$$\mathcal{L}_{\text{conn}}^{(i,j)} = -\tau_{\text{c}} \left(\log \sum_{u \in S_i,\, v \in S_j} \exp\!\left(-\frac{d_{uv,t}}{\tau_{\text{c}}} \right) - \log(n_i n_j) \right), \tag{5}$$

with temperature $\tau_{\text{c}} = 50\,\text{m}$, averaged over all $\binom{C_t}{2}$ component pairs. Without the $-\log(n_i n_j)$ correction, standard log-sum-exp systematically under-penalizes large components by $\tau_{\text{c}} \log(n_i n_j)$ and produces spuriously negative losses (verified numerically in Appendix A). Area coverage is approximated over Monte Carlo grid samples G_{mc} via $\mathcal{L}_{\text{cov}} = \frac{1}{|G_{\text{mc}}|} \sum_{\mathbf{g} \in G_{\text{mc}}} \big(1 - \sigma\big((R_{\text{cov}} - \min_i \|\mathbf{g} - \mathbf{p}_{i,t}\|_2)/\tau_{\text{v}}\big)\big)$ with $\tau_{\text{v}} = 10\,\text{m}$, and movement regularization is $\mathcal{L}_{\text{move}} = \frac{1}{|V_t|} \sum_i \|\mathbf{p}_{i,t+1} - \mathbf{p}_{i,t}\|_2$. The per-step loss is $\mathcal{L}_t = 0.5\, \mathcal{L}_{\text{conn}} + 0.5\, \mathcal{L}_{\text{cov}} + \lambda\, \mathcal{L}_{\text{move}}$ with $\lambda = 0.01$. Algorithms 1 and 2 detail the training and inference procedures.

6 Theoretical Analysis

6.1 Local Monotonicity of the Heuristic Branch

The heuristic virtual-force branch provides a certifiable local descent direction on inter-node distance within the attractive zone. We first analyze the test-time stability provided by the parameter-free branch.

Proposition 1 (Attractive-zone descent guarantee)**.** *Let v_i, v_j be two surviving UAVs at distance $d_u < d_{ij,t} \le R_c$ in the attractive zone of Eq. (2). If no repulsive neighbor forces or boundary forces act on v_i and v_j, then applying the heuristic velocity alone ($w_E = 1, w_T = 0$) with step size $0 < \Delta t < d_{ij,t}/(2\hat{v}_{\max})$ strictly decreases their Euclidean separation: $d_{ij,t+1} = d_{ij,t} - 2\Delta t\, \hat{v}_{\max} \frac{\|\mathbf{F}'_{ij,t}\|_2}{\|\mathbf{F}'_{ij,t}\|_2 + \epsilon} < d_{ij,t}$.*

Proof. By Eq. (2), $\mathbf{F}'_{ij,t} = m_{ij}\, \mathbf{u}_{ij,t}$ with magnitude $m_{ij} = \exp\!\big(-\beta \frac{R_c - d_{ij,t}}{R_c - d_u}\big) > 0$ along $\mathbf{u}_{ij,t} = (\mathbf{p}_{j,t} - \mathbf{p}_{i,t})/d_{ij,t}$, while by anti-symmetry $\mathbf{F}'_{ji,t} = -m_{ij}\, \mathbf{u}_{ij,t}$. Under the isolation condition, $\mathbf{v}'_{i,t} = \tilde{v}\, \mathbf{u}_{ij,t}$ and $\mathbf{v}'_{j,t} = -\tilde{v}\, \mathbf{u}_{ij,t}$ where $\tilde{v} = \hat{v}_{\max} \frac{m_{ij}}{m_{ij} + \epsilon} \in (0, \hat{v}_{\max})$. Updating positions yields $\mathbf{p}_{i,t+1} - \mathbf{p}_{j,t+1} = (\mathbf{p}_{i,t} - \mathbf{p}_{j,t}) + 2\Delta t\, \tilde{v}\, \mathbf{u}_{ij,t} = -(d_{ij,t} - 2\Delta t\, \tilde{v})\mathbf{u}_{ij,t}$. Taking the Euclidean norm gives $d_{ij,t+1} = d_{ij,t} - 2\Delta t\, \tilde{v} < d_{ij,t}$. □

Remark 1 (Safety margin under multi-neighbor interference and branch fusion)**.** *In dense swarms, competing neighbor forces and the learned velocity $\mathbf{v}''_{i,t}$ perturb this descent direction. Because the numerator of Eq. (4) is the convex combination $w_E(t)\mathbf{v}'_{i,t} + w_T(t)\mathbf{v}''_{i,t}$ of two vectors bounded in norm by $\hat{v}_{\max}$, preserving a non-negative projection along $\mathbf{v}'_{i,t}$ whenever $\mathbf{v}''_{i,t}$ opposes $\mathbf{v}'_{i,t}$ requires maintaining $w_E(t)$ above a state-dependent safety margin $w_E^{\min} \in (0, 0.5]$, or equivalently keeping $w_T(t)$ below $\tau = 1 - w_E^{\min} \in [0.5, 1)$.*

6.2 Fusion Schedule as an Implicit Training Curriculum

During end-to-end training, the sigmoid slope k governs how long the learned branch operates in an unassisted, high-autonomy regime. While Remark 1 explains why handing full control to an imperfect learned policy can disrupt local links at test time, it does not by itself explain why a steep adaptive schedule impairs the learned weights θ during training. Because π_θ is trained through the fused rollout of Algorithm 1, the schedule $w_T(t; k, s_0)$ determines the supervision regime at every rollout step.

Definition 1 (High-autonomy exposure)**.** *For a training rollout of length T_{train} and an autonomy threshold $\tau = 1 - w_E^{\min} \in (0.5, 1)$, the* high-autonomy exposure $\mathrm{AE}(k, s_0; \tau)$ *of schedule $w_T(\cdot; k, s_0)$ is the fraction of training steps in which the learned branch exceeds weight τ:*

$$\mathrm{AE}(k, s_0; \tau) = \frac{1}{T_{\text{train}}} \Big| \Big\{ t \in \{0, \dots, T_{\text{train}} - 1\} : w_T(t; k, s_0) > \tau \Big\} \Big|. \tag{6}$$

Inverting Eq. (3) shows that $w_T(t; k, s_0) > \tau$ if and only if

$$\frac{t}{T_{\text{train}}} > s_0 + \frac{1}{k} \ln\!\left(\frac{\tau}{1 - \tau}\right), \tag{7}$$

and the transition interval from $w_T = 0.1$ to $w_T = 0.9$ spans width $\Delta s_{10 \to 90} = \frac{\ln(81)}{k} \approx \frac{4.394}{k}$ of the rollout horizon. Equation (7) reveals the exact coupling between steepness k and midpoint s_0: for any safety threshold $\tau > 0.5$ (where $\ln \frac{\tau}{1-\tau} > 0$), reducing k directly delays the threshold

crossing and widens $\Delta s_{10\to 90}$. Under R2C's steep default ($k = 30, s_0 = 0.125$), the transition window $\Delta s_{10\to 90}$ spans only 14.6% of the horizon and AE$(30, 0.125; 0.8)$ reaches 82.5% of the training rollout, forcing a randomly initialized GCN to control the swarm almost solo from step $t = 7$ onward and compounding open-loop spatial collapse. Conversely, a smooth schedule ($k = 5, s_0 = 0.7$) expands $\Delta s_{10\to 90}$ to 87.9% of the rollout and reduces AE$(5, 0.7; 0.8)$ to 0.0%, keeping the learned branch in a well-conditioned residual-correction regime throughout training.

7 Experimental Setup

Simulation testbed and failure protocols follow R2C's physical parameters across random and uniform deployments. We evaluate all methods in a $1000 \times 1000\,\mathrm{m}^2$ mission square under two initial swarm layouts: **random deployment** with 200 UAVs sampled uniformly at random, and **uniform deployment** with 196 UAVs placed on a 14×14 regular grid. Following Peng et al. (2026), each UAV has communication and coverage radii $R_c = R_{\mathrm{cov}} = 120\,\mathrm{m}$, maximum speed $\hat{v}_{\max} = 1.0\,\mathrm{m/s}$, step size $\Delta t = 1.0\,\mathrm{s}$, and evaluation horizon $T = 400$ steps. At $t_1 = 0$, a failure ratio $r \in \{20\%, 40\%, 60\%, 80\%\}$ of UAVs is removed uniformly at random. Each primary architecture is trained across 5 independent random seeds (`seed` $\in \{1, 2, 3, 4, 5\}$) and evaluated over 50 independent failure trials per seed.

Baselines and evaluation metrics quantify recovery reliability, speed, coverage, and clustering. We compare **HAC-Fixed** and **HAC-Smooth** ($k = 5, s_0 = 0.7$) against five baselines: (i) **GAC-Fixed** (global learned GCN), (ii) **SAC-Fixed** (subnetwork rigid-translation GCN), (iii) **Learned-only** ($w_T(t) \equiv 1.0$ ablation of HAC), (iv) **Heuristic-only** ($w_E(t) \equiv 1.0$ virtual-force branch), and (v) published reference numbers from **R2C** (Peng et al., 2026). We report four metrics: **Success/50** (number of trials out of 50 achieving $C_{t_r} = 1$ within $T = 400$ steps), **REC** (recovery time ratio T_{rec} averaged over successful trials), **COV** (coverage ratio $C_{\mathrm{cov}}(t_r)$ at recovery), and **AND** (trajectory-averaged node degree up to t_r, measuring spatial clustering).

Implementation details and strict multi-seed control ensure statistical reproducibility. All learned backbones use $L_{\mathrm{gcn}} = 4$ GCN layers, hidden width 128, ReLU activations, and a 2-layer MLP head, trained for 300 epochs using Adam (learning rate 10^{-3}, gradient norm clip 1.0) over $T_{\mathrm{train}} = 40$ rollout steps with 800 Monte Carlo coverage grid points per step. Both NumPy environment RNGs and PyTorch CPU/CUDA weight initializations are strictly seeded prior to model instantiation (fixing an early unseeded-initialization bias documented in Appendix A).

8 Results

8.1 Main Results: HAC vs. Single-Branch Baselines

HAC-Fixed and HAC-Smooth achieve near-perfect recovery across all failure regimes, outperforming single-branch baselines by up to 36.8/50 trials (73.6 pp) at 80% node loss. Tables 3 and 4 report the evaluation results under random and uniform deployments, and Figure 1 illustrates the success-rate trajectories under random deployment. Across 20% to 60% failure rates in both geometries, HAC-Fixed achieves a 100% recovery rate (50.0 ± 0.0 out of 50 trials) with zero cross-seed variance. At the extreme 80% failure rate under random deployment, HAC-Fixed succeeds in 48.8 ± 0.98 trials and HAC-Smooth succeeds in 48.0 trials, surpassing the strongest single-branch baseline GAC-Fixed by 29.0 trials (58.0 pp) and the Learned-only ablation by 36.8 trials (73.6 pp), while SAC-Fixed and Heuristic-only drop to zero recoveries. Simultaneously,

Table 3: Main results under random deployment (mean ± std over 5 seeds for primary multi-seed rows; single-seed verification for HAC-Smooth). Success is out of 50 trials; n/a indicates zero successful trials.

Architecture	Failure	Success/50	REC	COV	AND
HAC-Fixed	20%	50.0 ± 0.0	0.061 ± 0.006	0.997 ± 0.000	6.72 ± 0.02
	40%	50.0 ± 0.0	0.136 ± 0.006	0.988 ± 0.001	5.29 ± 0.02
	60%	50.0 ± 0.0	0.307 ± 0.006	0.923 ± 0.005	3.97 ± 0.02
	80%	$\mathbf{48.8 \pm 0.98}$	0.666 ± 0.032	0.604 ± 0.005	2.58 ± 0.02
HAC-Smooth $(5, 0.7)$	20%	50.0	0.058	0.997	6.68
	40%	50.0	0.131	0.989	5.24
	60%	50.0	0.298	0.928	3.91
	80%	**48.0**	**0.652**	**0.619**	**2.46**
GAC-Fixed	20%	49.0 ± 0.6	0.324 ± 0.007	0.747 ± 0.009	13.68 ± 0.28
	40%	47.2 ± 1.7	0.467 ± 0.020	0.614 ± 0.021	12.48 ± 0.61
	60%	42.8 ± 3.7	0.612 ± 0.027	0.452 ± 0.022	10.55 ± 0.78
	80%	19.8 ± 5.5	0.775 ± 0.017	0.310 ± 0.009	6.00 ± 0.26
SAC-Fixed	20%	40.0 ± 0.0	0.216 ± 0.000	0.910 ± 0.000	6.82 ± 0.00
	40%	26.0 ± 0.0	0.567 ± 0.000	0.713 ± 0.000	5.85 ± 0.00
	60%	5.0 ± 0.0	0.852 ± 0.000	0.531 ± 0.000	4.18 ± 0.00
	80%	0.0 ± 0.0	n/a	n/a	n/a
Learned-only	20%	44.4 ± 1.4	0.277 ± 0.013	0.851 ± 0.008	10.24 ± 0.19
	40%	37.2 ± 1.6	0.402 ± 0.009	0.759 ± 0.008	9.07 ± 0.22
	60%	31.4 ± 3.2	0.613 ± 0.021	0.558 ± 0.019	8.11 ± 0.31
	80%	12.0 ± 3.5	0.874 ± 0.025	0.306 ± 0.016	5.90 ± 0.28
Heuristic-only	20%	47.4 ± 1.4	0.051 ± 0.017	0.997 ± 0.000	6.65 ± 0.04
	40%	30.4 ± 2.2	0.120 ± 0.027	0.993 ± 0.001	5.13 ± 0.05
	60%	1.4 ± 1.4	0.224 ± 0.091	0.964 ± 0.008	3.78 ± 0.12
	80%	0.0 ± 0.0	n/a	n/a	n/a

HAC-Fixed and HAC-Smooth nearly double the post-recovery coverage of GAC-Fixed and Learned-only at 80% failure (0.604–0.619 vs. 0.306–0.310) and cut recovery time by 14% to 25%.

Uniform grid deployments confirm the same architectural ranking and low variance. As shown in Table 4, HAC-Fixed maintains 100% recovery through 60% failure and reaches 49.0±2.0 out of 50 at 80% failure under uniform deployment, exceeding GAC-Fixed by 23.2 trials (46.4 pp) and Learned-only by 37.4 trials (74.8 pp) while doubling their post-recovery coverage (0.606 vs. 0.300–0.337). Both SAC-Fixed and Heuristic-only collapse to zero recoveries at 80% failure across both deployment topologies, demonstrating that their failure modes are structural consequences of single-branch operation rather than artifacts of initial node placement.

8.2 Comparison with Published R2C Results

Without boundary hulls or multipartite graph convolutions, HAC retains 88.7% to 90.7% of R2C's reported coverage at 80% failure (within 6.2 to 7.7 pp). Table 5 compares HAC-Fixed and HAC-Smooth against R2C's published metrics (Peng et al., 2026). Because R2C's source code is unavailable, this comparison is indicative rather than a same-codebase reproduction. From 20% to 60% failure, HAC matches R2C's coverage within 0.1 to 2.8 pp and achieves faster recovery at 20% and 40% uniform failure. At 80% failure, R2C's reported coverage exceeds HAC-Fixed by 7.7 pp under random deployment (0.681 vs. 0.604, or 88.7% relative retention) and by 6.2 pp under uniform deployment (0.668 vs. 0.606, or 90.7% relative retention), while HAC-Smooth narrows the random-deployment gap to 6.2 pp (0.681 vs. 0.619, or 90.9% relative retention). Considering that R2C employs Quickhull boundary extraction, cross-subgraph virtual edges, and an

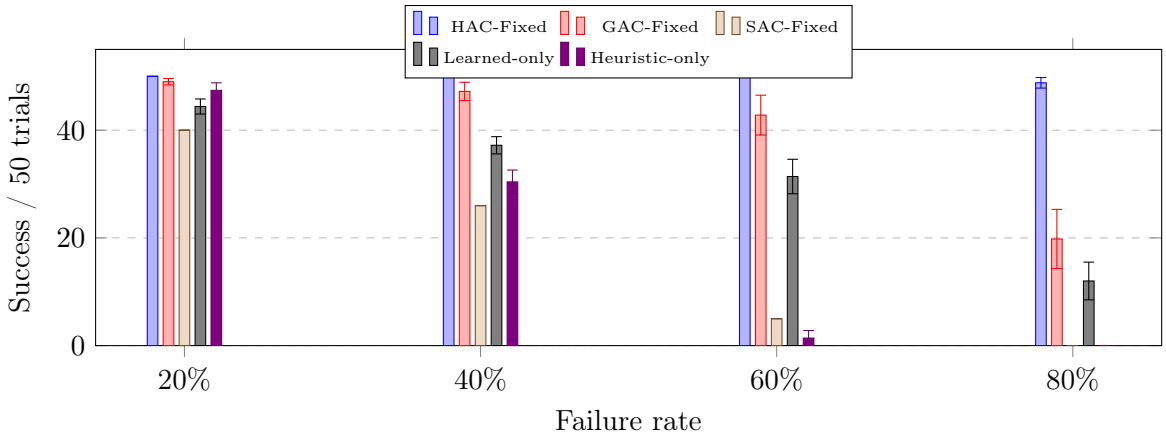


Figure 1: Connectivity recovery success count out of 50 trials across failure rates under random deployment (mean ± std over 5 seeds). HAC-Fixed sustains near-complete recovery with minimal variance even at 80% failure.

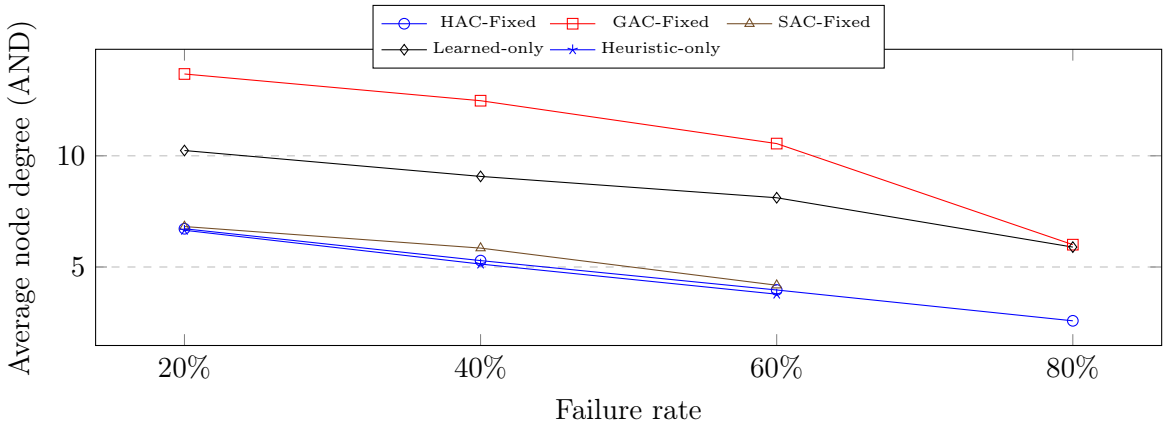


Figure 2: Trajectory-averaged node degree (AND) vs. failure rate under random deployment. HAC-Fixed matches the low clustering profile of SAC-Fixed and Heuristic-only while avoiding their high-failure connectivity collapse.

8-layer, 512-width Chebyshev spectral GCN whereas HAC uses a single 4-layer, 128-width global k-NN GCN, these results demonstrate that velocity-level hybridization itself captures roughly 90% of the coverage benefit.

8.3 Topology Quality Analysis

HAC prevents GNN-induced spatial clustering while sustaining full-swarm reconnection. Figure 2 plots the trajectory-averaged node degree (AND) across failure rates under random deployment. Whereas GAC-Fixed and Learned-only produce heavily clustered topologies (8.11–13.68 neighbors), HAC-Fixed and HAC-Smooth track the low, smoothly decreasing degree profile of the non-clustering Heuristic-only and SAC-Fixed baselines (6.72 down to 2.58), confirming that velocity-level virtual-force repulsion counteracts GNN aggregation collapse throughout the recovery trajectory.

Table 4: Main results under uniform grid deployment (mean ± std over 5 seeds).

Architecture	Failure	Success/50	REC	COV
HAC-Fixed	20%	50.0 ± 0.0	0.003 ± 0.000	1.000 ± 0.000
	40%	50.0 ± 0.0	0.037 ± 0.002	0.998 ± 0.000
	60%	50.0 ± 0.0	0.258 ± 0.012	0.957 ± 0.003
	80%	$\mathbf{49.0 \pm 2.0}$	0.641 ± 0.052	$\mathbf{0.606 \pm 0.005}$
GAC-Fixed	20%	50.0 ± 0.0	0.011 ± 0.003	0.992 ± 0.003
	40%	50.0 ± 0.0	0.200 ± 0.014	0.843 ± 0.019
	60%	47.0 ± 2.8	0.574 ± 0.013	0.478 ± 0.023
	80%	25.8 ± 6.1	0.772 ± 0.057	0.300 ± 0.012
SAC-Fixed	20%	49.8 ± 0.4	0.016 ± 0.006	0.996 ± 0.002
	40%	35.2 ± 3.7	0.256 ± 0.060	0.944 ± 0.017
	60–80%	0.0 ± 0.0	n/a	n/a
Learned-only	20%	50.0 ± 0.0	0.008 ± 0.003	0.998 ± 0.002
	40%	49.2 ± 0.7	0.228 ± 0.022	0.880 ± 0.021
	60%	33.8 ± 6.4	0.625 ± 0.012	0.581 ± 0.032
	80%	11.6 ± 6.5	0.866 ± 0.070	0.337 ± 0.024
Heuristic-only	20%	49.8 ± 0.4	0.006 ± 0.002	1.000 ± 0.000
	40%	44.0 ± 1.3	0.077 ± 0.017	0.998 ± 0.000
	60%	0.8 ± 0.7	0.110 ± 0.081	0.977 ± 0.009
	80%	0.0 ± 0.0	n/a	n/a

Table 5: Comparison between HAC and R2C's published numbers (Peng et al., 2026). At 80% failure, HAC-Fixed comes within 7.7 pp (random) and 6.2 pp (uniform) of R2C's coverage while omitting subnetwork boundary extraction and multipartite graph construction.

	Random Deployment		Uniform Deployment	
Failure	R2C REC / COV	HAC-Fixed REC / COV	R2C REC / COV	HAC-Fixed REC / COV
20%	0.023 / 0.996	0.061 / 0.997	0.037 / 1.000	0.003 / 1.000
40%	0.070 / 0.989	0.136 / 0.988	0.051 / 0.999	0.037 / 0.998
60%	0.204 / 0.951	0.307 / 0.923	0.181 / 0.978	0.258 / 0.957
80%	0.625 / **0.681**	0.666 / **0.604**	0.617 / **0.668**	0.641 / **0.606**

9 Ablation Studies

9.1 Branch Contribution Synergy

Velocity fusion yields super-additive recovery gains that exceed the sum of isolated branches at severe failure rates. Comparing HAC-Fixed against its isolated constituents in Tables 3 and 4 rules out simple branch selection. At 60% uniform failure, Heuristic-only recovers in 0.8 trials and Learned-only in 33.8 trials, whereas HAC-Fixed recovers in all 50.0 trials. At 80% random failure, Heuristic-only (0.0) and Learned-only (12.0) sum to 12.0 recoveries, yet fusing them reaches 48.8 recoveries. The global GCN pulls distant components into communication range R_c, where the virtual-force branch's attractive zone (Proposition 1) locks links in place while its repulsive zone prevents over-concentration.

9.2 Adaptive Schedule Steepness and Autonomy Exposure

Smoothing the sigmoid transition slope k reduces high-autonomy exposure during training and restores 80%-failure recovery from 20/50 to 48/50. Directly applying R2C's steep schedule ($k = 30, s_0 = 0.125$) to HAC degrades 80%-failure recovery to 20/50 under random

Table 6: Schedule steepness sweep under random deployment showing theoretical transition width $\Delta s_{10\to 90}$, high-autonomy training exposure $\mathrm{AE}(k, s_0; 0.8)$, and empirical recovery counts out of 50 trials.

Schedule (k, s_0)	$\Delta s_{10\to 90}$	$\mathrm{AE}(k, s_0; 0.8)$	20%	40%	60%	80%
R2C Default $(30, 0.125)$	14.6%	82.5%	50	50	45	20
Intermediate $(10, 0.125)$	43.9%	72.5%	50	50	46	17
Delayed Steep $(30, 0.5)$	14.6%	45.0%	50	50	43	8
Delayed Smooth $(10, 0.5)$	43.9%	35.0%	50	50	49	34
HAC-Smooth $(5, 0.7)$	**87.9%**	**0.0%**	**50**	**50**	**50**	**48**

Table 7: Counter-experiment with HSC (heuristic branch + subnetwork-level rigid-translation GCN). Unlike HAC, schedule smoothing cannot overcome HSC's lack of node-level expressiveness at 80% failure.

	Random (Success/50)		Uniform (Success/50)	
Variant	60%	80%	60%	80%
HSC-Fixed	40	0	42	0
HSC-Adaptive $(30, 0.125)$	16	0	13	0
HSC-Smooth $(5, 0.7)$	33	1	40	0

deployment and 7/50 under uniform deployment. Table 6 sweeps (k, s_0) alongside the theoretical transition width $\Delta s_{10\to 90} = \frac{\ln 81}{k}$ and high-autonomy exposure $\mathrm{AE}(k, s_0; 0.8)$ computed via Eq. (6). Steep transitions ($k = 30$) yield narrow handover windows ($\Delta s_{10\to 90} = 14.6\%$) and poor 80%-failure recovery regardless of whether the transition occurs early ($s_0 = 0.125$, 20/50) or mid-rollout ($s_0 = 0.5$, 8/50). Widening the transition window and lowering $\mathrm{AE}(k, s_0; 0.8)$ monotonically improves recovery across the $s_0 \geq 0.5$ regimes, culminating in **HAC-Smooth** ($k = 5, s_0 = 0.7$), where $\Delta s_{10\to 90} = 87.9\%$ and $\mathrm{AE}(5, 0.7; 0.8) = 0.0\%$ achieve 48/50 random-deployment recoveries (and 37/50 vs. 7/50 under uniform deployment) while improving coverage (0.619 vs. 0.604) and lowering AND (2.46 vs. 2.58) relative to HAC-Fixed.

9.3 Granularity Substitution: HSC as a Counter-Experiment

Schedule smoothing cannot rescue subnetwork-level rigid-translation policies that lack node-level velocity expressiveness. To test whether schedule smoothing is architecture-agnostic, we evaluate **HSC**, which fuses the same virtual-force heuristic branch with SAC's subnetwork-level rigid-translation GCN under Eq. (4). As shown in Table 7, HSC-Fixed fails completely at 80% node loss across both deployments (0/50), and applying the smoothed schedule ($k = 5, s_0 = 0.7$) yields almost zero recovery at 80% failure (1/50 random, 0/50 uniform). This counter-experiment cleanly separates two distinct failure mechanisms: HAC's degradation under R2C's default schedule is a *curriculum scheduling* failure curable by smoothing k, whereas HSC's collapse is an *architectural expressiveness* failure caused by restricting subnetwork motion to rigid translation without boundary deformation.

10 Discussion

10.1 Underlying Physical and Optimization Mechanisms

Complementary receptive fields and residual curriculum supervision explain the effectiveness of simple global hybrids. Across our experiments, the heuristic virtual-force branch and the global k-NN GCN address orthogonal spatial scales: the global GCN supplies long-range attraction gradients across disconnected components separated beyond R_c, while the virtual-force branch enforces local collision buffering ($d < d_l$) and certifiable link capture ($d_u < d \leq R_c$, Proposition 1). When trained end-to-end through Eq. (4), maintaining a non-trivial heuristic weight via either fixed 50/50 fusion or a smooth sigmoid schedule ($k = 5$) stabilizes forward rollouts and allows the GCN to learn smooth residual corrections rather than fighting open-loop trajectory divergence.

10.2 Practical Deployment Guidelines

Practitioners should match fusion schedules to policy expressiveness and expected failure severity. Our findings translate into three concrete design guidelines for resilient multi-UAV controllers:

- **Default to HAC-Fixed for maximum cross-topology reliability:** Fixed equal weighting ($w_E = w_T = 0.5$) requires zero schedule hyperparameter tuning and achieves 97.6% to 100% recovery across all failure ratios in both random and grid deployments.
- **Use HAC-Smooth ($k = 5, s_0 = 0.7$) when post-recovery coverage uniformity is paramount:** A gentle transition slope slightly improves post-recovery coverage (0.619 vs. 0.604) and reduces node clustering (AND 2.46 vs. 2.58) under severe random failures while preserving high recovery rates.
- **Ensure node-level output expressiveness before applying adaptive handovers:** Subnetwork-level rigid translation (SAC, HSC) should only be used under light failure regimes ($\leq 40\%$) unless augmented with per-node residual velocities or boundary-node deformation.

10.3 Limitations and Future Work

Closing the remaining 6.2 to 7.7 pp high-failure coverage gap and scaling training horizons remain open directions. First, while HAC retains roughly 90% of R2C's reported coverage at 80% failure, isolating whether R2C's remaining 6.2–7.7 pp advantage stems from Quickhull boundary weighting, multipartite virtual edges, or its 16× wider spectral GCN requires a controlled re-implementation of those sub-modules inside our simulator. Second, all learned policies are trained on $T_{\text{train}} = 40$ rollout steps for backpropagation-through-time tractability and evaluated over $T = 400$ steps; curriculum horizon scaling or truncated BPTT over longer rollouts could further improve late-stage coverage. Third, running multi-seed schedule sweeps and decoupling training-time vs. evaluation-time fusion schedules (supported directly in our released scripts) will provide tighter statistical bounds on intermediate (k, s_0) configurations.

11 Conclusion

We systematically deconstructed the hybrid design space for UAV swarm connectivity recovery along decision paradigm, spatial reasoning granularity, and trade-off scheduling. By pairing a parameter-free virtual-force branch with a single global k-NN graph convolution branch, HAC achieves near-perfect connectivity restoration across 20% to 80% node failures, outperforms single-branch learned baselines by up to 73.6 pp in recovery rate, and retains 88.7% to 90.7% of R2C's reported coverage

without subnetwork partitioning or convex-hull boundary extraction. Furthermore, we demonstrated theoretically and empirically that adaptive fusion schedules act as an implicit training curriculum whose transition steepness k must remain smooth to prevent high-autonomy rollout divergence, and proved via a subnetwork-level counter-experiment (HSC) that schedule smoothing only succeeds when the learned policy possesses node-level velocity expressiveness.

References

Ameer A. Abbasi and Mohamed Younis. Movement-assisted connectivity restoration in wireless sensor and actor networks. *IEEE Transactions on Parallel and Distributed Systems*, 20(9):1366–1379, 2008.

Kemal Akkaya, Fatih Senel, Aravind Thimmapuram, and Suleyman Uludag. Distributed recovery from network partitioning in movable sensor/actor networks via controlled mobility. *IEEE Transactions on Computers*, 59(2):258–271, 2009.

C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa. The quickhull algorithm for convex hulls. *ACM Transactions on Mathematical Software*, 22(4):469–483, 1996.

Ming Chen, Hao Wang, Chih-Yung Chang, and Xin Wei. SIDR: A swarm intelligence-based damage-resilient mechanism for UAV swarm networks. *IEEE Access*, 8:77089–77105, 2020.

Andrea Coletta, Francesco Giorgi, Gaia Maselli, Marco Prata, Daniel Silvestri, Jonathan Ashdown, and Francesco Restuccia. A 2-UAV: Application-aware content and network optimization of edge-assisted UAV systems. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pages 1–10, 2023.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.

Zhiwei Guo, Yifan Sun, Yue Wang, Chen Fu, and Jing Zhong. An intelligent framework for resilience recovery of FANETs with spatio-temporal aggregation and multi-head attention mechanism. *Computers, Materials & Continua*, 79(2):2375–2398, 2024.

Yogesh K. Joshi. Autonomous recovery from multi-node failure in wireless sensor networks. In *2012 IEEE Global Communications Conference (GLOBECOM)*, pages 652–657, 2012.

Nikolaos Karalias and Andreas Loukas. Erdos goes neural: An unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems*, 33: 6659–6672, 2020.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Harrshavasan Kurunathan, Hailong Huang, Kai Li, Wei Ni, and Ekram Hossain. Machine learning-aided operations and communications of unmanned aerial vehicles: A contemporary survey. *IEEE Communications Surveys & Tutorials*, 26(1):496–533, 2023.

Jian Li, Ping Yi, Tong Duan, Ze Zhang, Jing Li, Yue Wang, and Jian Yu. Fast connectivity restoration of UAV communication networks based on distributed hybrid MADDPG and APF algorithm. *Ad Hoc Networks*, 171:103785, 2025.

Jun Li, Cheng Zhou, Jia Liu, Min Sheng, Nan Zhao, and Yi Su. Reinforcement learning-based resource allocation for coverage continuity in high dynamic UAV communication networks. *IEEE Transactions on Wireless Communications*, 23(2):848–860, 2023.

Hao Lin and Liang Ding. Multi-hop differential topology based algorithms for resilient network of UAV swarm. *arXiv preprint arXiv:2411.11342v2*, 2025.

Hao Lin, Liang Ding, Song Chen, Fan Yang, and Ling Qian. Multi-hop diffused graph convolution for resilient UAV swarm networks. In *2024 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB)*, pages 1–6, 2024.

Fang Liu and Qing Wang. Asymmetric learning for spectral graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 18798–18806, 2025.

Zhangfeng Ma, Bo Ai, Ruisi He, Guangjie Wang, Yong Niu, and Zhangdui Zhong. A wideband non-stationary air-to-air channel model for UAV communications. *IEEE Transactions on Vehicular Technology*, 69(2):1214–1226, 2019.

Zhenqiang Mi and Jenny Y. Yang. Obstacle-avoidance connectivity restoration for mobile sensor systems with local information. In *2015 IEEE International Conference on Communications (ICC)*, pages 6395–6399, 2015.

Zhenqiang Mi, Yang Yang, and Guangming Liu. HERO: A hybrid connectivity restoration framework for mobile multi-agent networks. In *2011 IEEE International Conference on Robotics and Automation*, pages 1702–1707, 2011.

Zhiyu Mou, Feifei Gao, Jian Liu, and Qing Wu. Resilient UAV swarm communications with graph convolutional neural network. *IEEE Journal on Selected Areas in Communications*, 40(1):393–411, 2022.

Dor Mutzari, Yonatan Aumann, and Sarit Kraus. Heterogeneous multi-robot graph coverage with proximity and movement constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 14646–14654, 2025.

Hao Ni, Qiuming Zhu, Bo Hua, Kai Mao, Yuxin Pan, Farman Ali, Weizhi Zhong, and Xiaomin Chen. Path loss and shadowing for UAV-to-ground UWB channels incorporating the effects of built-up areas and airframe. *IEEE Transactions on Intelligent Transportation Systems*, 25(11): 17066–17077, 2024.

Yabin Peng, Chang Liu, Shaoxun Liu, Yuan Liu, and Yue Wu. SmartTRO: Optimizing topology robustness for internet of things via deep reinforcement learning with graph convolutional networks. *Computer Networks*, 218:109385, 2022.

Yabin Peng, Jun Wu, Tong Duan, Yuan Liu, Ze Zhang, and Jie Zhang. Prioritized recovery strategy for robust UAV swarm communication via graph reinforcement learning. *IEEE Internet of Things Journal*, 12(13):23891–23904, 2025.

Yabin Peng, Chenyu Zhou, Hainan Cui, Tong Duan, Haoyang Chen, Fan Zhang, and Shaoxun Liu. Resilient UAV swarm with fast connectivity recovery and extensive coverage. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 917–925, 2026.

Jan Rychlewski. On hooke's law. *Journal of Applied Mathematics and Mechanics*, 48(3):303–314, 1984.

Ran Shi, Xin Yu, Yue Wang, Ye Tian, Zhi Liu, Wei Wu, Xiao-Ping Zhang, and Manuela M. Veloso. Symmetry-informed MARL: A decentralized and cooperative UAV swarm control approach for communication coverage. *IEEE Transactions on Mobile Computing*, 24(9):8039–8056, 2025.

Siddharth Shriwastav. Round-table negotiation for fast restoration of connectivity in partitioned wireless sensor networks. *Ad Hoc Networks*, 77:11–27, 2018.

Mehmet Tosun, Umut Can Cabuk, Emre Haytaoglu, Orhan Dagdeviren, and Yusuf Ozturk. DP-kCR: Distributed proactive k-connectivity recovery algorithm for UAV-based MANETs. *IEEE Transactions on Reliability*, 73(4):1918–1932, 2024.

Jian Wang, Shuo Li, Di Chen, Qiang Fu, Qi Qi, Hao Sun, Bo He, and Jianxin Liao. Flight trajectory control with network-oriented hierarchical reinforcement learning for UAV-assisted time-sensitive IoT. *IEEE Transactions on Intelligent Transportation Systems*, 26(5):6332–6345, 2025a.

Xu Wang, Zhen Zhao, Liang Yi, Zhaolong Ning, Lei Guo, F. Richard Yu, and Song Guo. A survey on security of UAV swarm networks: Attacks and countermeasures. *ACM Computing Surveys*, 57(3):1–37, 2024a.

Yao Wang, Wei Chen, Tom H. Luan, Zhou Su, Qichao Xu, Ru Li, and Ning Chen. Task offloading for post-disaster rescue in unmanned aerial vehicle networks. *IEEE/ACM Transactions on Networking*, 30(4):1525–1539, 2022.

Yingze Wang, Zhen Feng, Hui Zhang, Yang Gao, Jing Lei, Le Sun, and Ming Song. Angle robustness unmanned aerial vehicle navigation in GNSS-denied scenarios. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 10386–10394, 2024b.

Yue Wang, Xu Wang, Chao Wei, Qi Ren, and Yong Tang. UAV swarm network topology self-healing via graph-based deep reinforcement learning. In *2025 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6, 2025b.

Zhi Wang, Xin Zhao, Jie Zhang, Nan Yang, Peng Wang, Jian Tang, Jun Zhang, and Liang Shi. APF-CPP: An artificial potential field based multi-robot online coverage path planning approach. *IEEE Robotics and Automation Letters*, 9(11):9199–9206, 2024c.

Chejian Xu, Wenhao Ding, Weijie Lyu, Zuxin Liu, Shuai Wang, Yihan He, Hanjiang Hu, Ding Zhao, and Bo Li. SafeBench: A benchmarking platform for safety evaluation of autonomous vehicles. *Advances in Neural Information Processing Systems*, 35:25667–25682, 2022.

Zhiqiang Ye, Ke Wang, Yu Chen, Xiang Jiang, and Guanghua Song. Multi-UAV navigation for partially observable communication coverage by graph reinforcement learning. *IEEE Transactions on Mobile Computing*, 22(7):4056–4069, 2022.

Dingyi Zeng, Wanlu Liu, Wenjie Chen, Liang Zhou, Miao Zhang, and Hua Qu. Substructure aware graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11129–11137, 2023.

Lei Zhang, Yi Du, Jing Xu, and Xin Wang. UAV-enabled IoT: Cascading failure model and topology-control-based recovery scheme. *IEEE Internet of Things Journal*, 11(12):22562–22577, 2024.

A Deviations and Bug-Fix Log

We document every implementation choice where our testbed deviates from Peng et al. (2026) or corrects numerical issues discovered during development:

- **Failure-node selection.** R2C specifies "unpredictable random failure patterns" without detailing the sampling distribution; we apply uniform random node removal without replacement.
- **Count-invariant connectivity surrogate loss.** R2C's discrete component count objective is non-differentiable. An initial soft-minimum estimator omitted the $-\log(n_i n_j)$ normalization in Eq. (5), causing large component pairs to produce spuriously negative losses purely due to pair-count summation bias. On a synthetic test with true minimum separation $\approx$ 100–108 m, the uncorrected estimator returned 2.85 for 3×3 nodes vs. -230.58 for 30×30 nodes, whereas our corrected Eq. (5) returned 112.71 and 109.54, respectively. All reported models use the corrected loss.
- **Boundary buffer width and clamping.** R2C leaves d_e unspecified; we set $d_e = 0.15R_c = 18$ m and enforce $0 \leq d_{i,b} < d_e$ so nodes clamped exactly onto mission boundaries continue to receive inward repulsion.
- **Rollout horizon and Monte Carlo coverage.** Differentiable training uses $T_{\text{train}} = 40$ steps with 800 uniform random grid points per step, whereas evaluation runs for $T = 400$ steps on a deterministic 5 m grid (40,000 points).
- **Weight-initialization seeding fix.** Early scripts seeded NumPy's environment RNG via `-seed` but left PyTorch weight initialization unseeded. An unseeded GAC-Fixed run yielded an optimistic 20%-failure COV of 0.830 and AND of 11.87, whereas properly seeding `torch.manual_seed` and `torch.cuda.manual_seed_all` across five seeds yielded COV 0.747 ± 0.009 and AND 13.68 ± 0.28 (Table 3). All multi-seed tables use strict full-stack seeding.

B Hyperparameter Summary

Table 8: Hyperparameters shared across all trained architectures unless otherwise noted.

Parameter	Value
GCN layers L_{gcn} / Hidden dimension	4 / 128
Euclidean k-NN degree k_{nn}	8
Optimizer / Learning rate / Gradient clip	Adam / 1×10^{-3} / 1.0
Training epochs / Training rollout T_{train}	300 / 40 steps
Evaluation horizon T / Step size Δt	400 steps / 1.0 s
Movement cost coefficient λ	0.01
Soft-min temperatures $\tau_{\text{c}}, \tau_{\text{v}}$	50.0 m, 10.0 m
Heuristic decay constants α, β, δ	0.01
Neutral zone $[d_l, d_u]$ / Boundary buffer d_e	$[0.8R_c, 0.9R_c]$ / $0.15R_c$
Communication & coverage radii $R_c = R_{\text{cov}}$	120 m
Maximum UAV speed $\hat{v}_{\max}$	1.0 m/s
Mission area $\mathcal{A}$	1000×1000 m^2
Swarm size (Random / Uniform 14×14)	200 / 196 UAVs

C Additional Uniform-Deployment Degree Statistics

For uniform grid deployment (20% → 40% → 60% → 80% failure), trajectory-averaged node degrees (AND) are: **HAC-Fixed** 5.72 → 4.32 → 3.30 → 2.37; **GAC-Fixed** 5.88 → 6.86 → 8.75 → 5.81; **SAC-Fixed** 5.74 → 4.46 → n/a → n/a; **Learned-only** 5.76 → 6.37 → 7.34 → 5.29; and **Heuristic-only** 5.72 → 4.36 → 2.87 → n/a.

D Code Structure and Full Reproduction Suite

The accompanying repository provides end-to-end training, evaluation, and multi-seed aggregation scripts for all architectures in Tables 2–7, including global (`HAC`, `GAC`, `Learned-only`) and subnetwork-level (`SAC`, `HSC`) backbones as well as train-vs.-evaluation schedule decoupling experiments.